KARMA Supplementary Appendix

1 KARMA Operator Descriptions

Table 1: KARMA operator descriptions

Operator	Output	Description
EntityLink $(q, \mathcal{G})$	E_q=$\{e_1, \ldots, e_k\}$	DBpedia Spotlight with recovery linking returns DBpedia resource IRIs; these entities seed schema projection, SPARQL generation, and entity-centered text retrieval.
SchemaProject (E_q, q)	$\mathcal{S}_q$=$\langle P_q, A_q \rangle$	Builds the projected schema for q: P_q contains instantiated (e, p) pairs used as coverage evidence, while A_q stores annotation metadata such as class, range, type, or datatype fields. Bridge candidates may be shown in traces, but coverage still depends on instantiated KG evidence, not on annotations alone.
Covers $(\mathcal{S}_q, q)$	**true**/**false**; covered, uncovered; next branch	Checks whether each relation in Rel(q) has a coverage-compatible predicate in P_q on the required linked entity. The reported covered and uncovered relations determine whether the algorithm continues with SPARQL, decomposition, or text retrieval.
Text2SPARQL $(q, \mathcal{S}_q, \mathcal{G})$	SPARQL query, bindings R, answer a or $\emptyset$	A schema-grounded prompt asks the LLM to generate a SPARQL query from q and $\mathcal{S}_q$; the KG returns bindings and any answer. Non-empty answers return or enter M; empty results leave the step unresolved.
Decompose (Q)	D=$(\{q_i\}, \prec)$	A decomposition prompt asks the LLM to return a dependency-ordered sub-question DAG. The order tells the system which sub-answers must be produced before later questions can be grounded.
Ground (q_i, M)	$\hat{q}_i$	Deterministic substitution produces a concrete sub-question by replacing dependencies with answers already stored in M.
TextRAG $(q, E_q, \mathcal{T})$	relevant passages, answer a or a_i	Retrieval supplies entity-seeded passages when available; an answer prompt asks the LLM to answer q using those passages after failed coverage, empty SPARQL results, or empty synthesis.
M and Synthesize (Q, M)	$M[q_i] \leftarrow a_i$; final answer a	M records sub-question answers. A synthesis prompt asks the LLM to combine M into the final answer; if it returns $\emptyset$, the algorithm calls final TextRAG.

As in Table 2, SchemaProject rows display $\mathcal{S}_q = \langle P_q, A_q \rangle$: P_q contains coverage-bearing instantiated predicates, while A_q contains annotations such as class metadata. Candidate bridge paths are shown only when retained for routing. Covers rows show the Boolean result, covered and uncovered relation checks, and the resulting branch.

Table 2: End-to-end trace of KARMA on Q="What did the spouse of Andrea Dotti win a Tony for?". SCHEMAPROJECT reports annotation metadata, such as class, and instantiated predicates; only instantiated predicates provide coverage evidence. The table reports the Boolean COVERS result and the branch subsequently taken by the KARMA algorithm. $\hat{q}$ denotes the instantiated form of a sub-question.

Step	Operator call	Output / state
Top level — COVERS fails, fall through to decomposition		
1	ENTITYLINK(Q)	E={`dbr:Andrea_Dotti_(psychiatrist)`}
2	SCHEMAPROJECT(E, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Person`; P_Q: {`dbo:spouse`, `dbo:birthDate`, ...} **false**; covered={*spouse* -> `dbo:spouse`}
3	COVERS($\mathcal{S}, Q$)	uncovered={*win a Tony for* -> no coverage-compatible instantiated predicate} next branch=**DECOMPOSE**
4	DECOMPOSE(Q)	D={q_1, q_2}, $q_1 \rightarrow q_2$; $M \leftarrow \emptyset$ q_1="Who is the spouse of Andrea Dotti?" q_2="What did [q_1] win a Tony for?"
*Iterate q_1 (M=$\emptyset$) ⇒ **KG-only***		
5	GROUND(q_1, M)	$\hat{q}_1$="Who is the spouse of Andrea Dotti?"
6	ENTITYLINK($\hat{q}_1$)	E_{q_1}={`dbr:Andrea_Dotti_(psychiatrist)`}
7	SCHEMAPROJECT($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Person`; P_{q_1}: {`dbo:spouse`, `dbo:birthDate`, ...} **true**; covered={*spouse* -> `dbo:spouse`}
8	COVERS($\mathcal{S}_{q_1}, \hat{q}_1$)	uncovered=$\emptyset$; next branch=**SPARQL**
9	TEXT2SPARQL	`SELECT ?s WHERE {dbr:Andrea_Dotti_(psychiatrist) dbo:spouse ?s}` $\rightarrow a_{q_1}$ = *Audrey Hepburn*; $M[q_1] \leftarrow$ *Audrey Hepburn*
*Iterate q_2 ⇒ **text** (coverage fails)*		
10	GROUND(q_2, M)	$\hat{q}_2$="What did Audrey Hepburn win a Tony for?"
11	ENTITYLINK($\hat{q}_2$)	E_{q_2}={`dbr:Audrey_Hepburn`} (`dbo:Person`)
12	SCHEMAPROJECT($E_{q_2}, \hat{q}_2$)	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2}: {`dbo:spouse`, `dbo:occupation`, ...} **false**; covered=$\emptyset$
13	COVERS($\mathcal{S}_{q_2}, \hat{q}_2$)	uncovered={*win a Tony for* -> no coverage-compatible instantiated predicate} next branch=**TEXT**
14	TEXTRAG($\hat{q}_2, E_{q_2}, \mathcal{T}$)	lead passage: "*Best Performance by a Leading Actress in a Play in Ondine*" $\rightarrow a_{q_2}$ = *Best Performance by a Leading Actress in a Play in Ondine*
15	SYNTHESIZE(Q, M)	*for Best Performance by a Leading Actress in a Play in Ondine*

2 End-to-end execution trace

Table 3: Trace-style examples of KARMA execution regimes selected from judged evaluation traces. Each block follows the same presentation pattern as Table 2: schema projection rows separate coverage-bearing predicates from annotations, and coverage rows report covered relations, uncovered relations, and the next branch. Text2SPARQL rows omit shared prefixes and abbreviate DBpedia IRIs with `dbr:`.

Step	Operator call	Output / state
Route: *Direct SPARQL* **Question:** "Where was the director of film *Thomas Jefferson (Film)* born?"		
1	EntityLink(Q)	E_Q={e_0=`dbr:Thomas_Jefferson_(film)`}.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Film`; P_Q: {`dbo:director`, ...}; bridge path: `dbo:director`→`dbo:birthPlace`.
3	Covers ($\mathcal{S}_Q, Q$)	**true**; covered={*director* -> `dbo:director`, *born* -> `dbo:director` → `dbo:birthPlace`}; uncovered=∅; next branch=**SPARQL**.

4	Text2SPARQL ($Q, \mathcal{S}_Q, \mathcal{G}$)	`SELECT ?birthPlace`
		`WHERE {`
		`dbr:Thomas_Jefferson_(film) dbo:director ?person .`
		`?person dbo:birthPlace ?birthPlace .`
		`}`
		`LIMIT 100`
		R={`dbr:Brooklyn`}.
		a=*Brooklyn*.

Route: *Direct SPARQL* **Question:** "Which film has the director born later, *House Of Blackmail* or *Taxi Zum Klo*?"		
1	EntityLink(Q)	E_Q={e_1, e_2}; e_1=`dbr:House_of_Blackmail`; e_2=`dbr:Taxi_zum_Klo`.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; for each $e \in E_Q$, A_Q: class=`dbo:Film`; P_Q: {`dbo:director`, ...}; bridge path: `dbo:director`→`dbo:birthDate`.
3	Covers ($\mathcal{S}_Q, Q$)	**true**; covered={*director* -> `dbo:director`, *born later* -> `dbo:director` → `dbo:birthDate`}; uncovered=∅; next branch=**SPARQL**.

4	Text2SPARQL ($Q, \mathcal{S}_Q, \mathcal{G}$)	`SELECT ?film`
		`WHERE {`
		`VALUES ?film { dbr:House_of_Blackmail dbr:Taxi_zum_Klo }`
		`?film dbo:director ?director .`
		`?director dbo:birthDate ?birthDate .`
		`}`
		`ORDER BY DESC(?birthDate)`
		`LIMIT 1`
		a=*Taxi Zum Klo*.

Route: *Decomp: all SPARQL* **Question:** "Who was born first out of Marco De Luigi and Fabio Maistro?"		
1	EntityLink(Q)	E_Q={e_1, e_2}; e_1=`dbr:Marco_de_Luigi`; e_2=`dbr:Fabio_Maistro`.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; for each $e \in E_Q$, A_Q: class=`dbo:Person`; P_Q: {`dbo:birthDate`, ...}.
3	Covers ($\mathcal{S}_Q, Q$)	covered={*born* -> `dbo:birthDate` for both entities}; uncovered={*born first* -> no accepted whole-question comparison route}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}, \emptyset$), ordered as q_1, q_2; q_1="When was Marco De Luigi born?"; q_2="When was Fabio Maistro born?"; M←∅.
5	Ground(q_1, M)	$\hat{q}_1$="When was Marco De Luigi born?".
6	EntityLink($\hat{q}_1$)	E_{q_1}={e_1=`dbr:Marco_de_Luigi`}.
7	SchemaProject ($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Person`; P_{q_1}: {`dbo:birthDate`, ...}.
8	Covers ($\mathcal{S}_{q_1}, \hat{q}_1$)	**true**; covered={*born* -> `dbo:birthDate`}; uncovered=∅; next branch=**SPARQL**.

9	Text2SPARQL ($\hat{q}_1, \mathcal{S}_{q_1}, \mathcal{G}$)	`SELECT ?birthDate`
		`WHERE {`
		`dbr:Marco_de_Luigi dbo:birthDate ?birthDate .`
		`}`
		`LIMIT 100`
		R={1978-03-21}; $M[q_1]$←*March 21, 1978*.

10	Ground(q_2, M)	$\hat{q}_2$="When was Fabio Maistro born?".
11	EntityLink($\hat{q}_2$)	E_{q_2}={e_2=`dbr:Fabio_Maistro`}.

continued on next page

Table 3: Trace-style examples of KARMA execution regimes (continued).

Step	Operator call	Output / state
12	SchemaProject $(E_{q_2}, \hat{q}_2)$	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2}: {`dbo:birthDate`, ...}.
13	Covers $(\mathcal{S}_{q_2}, \hat{q}_2)$	**true**; covered={*born* -> `dbo:birthDate`}; uncovered=$\emptyset$; next branch=**SPARQL**.

14	Text2SPARQL $(\hat{q}_2, \mathcal{S}_{q_2}, \mathcal{G})$	`SELECT ?birthDate`
		`WHERE {`
		`dbr:Fabio_Maistro dbo:birthDate ?birthDate .`
		`}`
		`LIMIT 100`
		R={1998-04-05}; $M[q_2] \leftarrow$ *April 5, 1998*.

15	Synthesize(Q, M)	LLM synthesis compares $M[q_1]$ and $M[q_2]$; a=*Marco De Luigi*.
Route: *Decomp: SPARQL+text* **Question:** "Where was the director of film *Age-Old Friends* born?"		
1	EntityLink(Q)	E_Q={e_0=`dbr:Age-Old_Friends`}.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Film`; P_Q: {`dbo:director`, ...}; candidate bridge path: `dbo:director`→`dbo:birthPlace`.
3	Covers $(\mathcal{S}_Q, Q)$	**false**; covered={*director* -> `dbo:director`}; uncovered={*born* -> unavailable bridge `dbo:director` → `dbo:birthPlace`}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}$, $q_1 \rightarrow q_2$); q_1="Who directed Age-Old Friends?"; q_2="Where was [q_1] born?"; $M \leftarrow \emptyset$.
5	Ground(q_1, M)	$\hat{q}_1$="Who directed Age-Old Friends?".
6	EntityLink$(\hat{q}_1)$	E_{q_1}={e_0}.
7	SchemaProject $(E_{q_1}, \hat{q}_1)$	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Film`; P_{q_1}: {`dbo:director`, ...}.
8	Covers $(\mathcal{S}_{q_1}, \hat{q}_1)$	**true**; covered={*directed* -> `dbo:director`}; uncovered=$\emptyset$; next branch=**SPARQL**.

9	Text2SPARQL $(\hat{q}_1, \mathcal{S}_{q_1}, \mathcal{G})$	`SELECT ?director`
		`WHERE {`
		`dbr:Age-Old_Friends dbo:director ?director .`
		`}`
		`LIMIT 100`
		R={`dbr:Allan_Kroeker`}; $M[q_1] \leftarrow$ *Allan Kroeker*.

10	Ground(q_2, M)	$\hat{q}_2$="Where was Allan Kroeker born?".
11	EntityLink$(\hat{q}_2)$	E_{q_2}={e_1=`dbr:Allan_Kroeker`}.
12	SchemaProject $(E_{q_2}, \hat{q}_2)$	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2} does not contain `dbo:birthPlace`.
13	Covers $(\mathcal{S}_{q_2}, \hat{q}_2)$	**false**; covered=$\emptyset$; uncovered={*born* -> no coverage-compatible `dbo:birthPlace`}; next branch=**TEXT**.
14	TextRAG $(\hat{q}_2, E_{q_2}, \mathcal{T})$	LLM answer over retrieved text; $\mathcal{T}_{q_2,\text{top}} \ni$ *Allan Kroeker*; a_{q_2}=*Winnipeg, Manitoba*; $M[q_2] \leftarrow$ *Winnipeg, Manitoba*.
15	Synthesize(Q, M)	LLM synthesis; a=*Winnipeg, Manitoba*.

continued on next page

Table 3: Trace-style examples of KARMA execution regimes (continued).

Step	Operator call	Output / state
Route: *Text only* **Question:** "Why did John Middleton Murry's wife die?"		
1	EntityLink(Q)	E_Q={e_0=`dbr:John_Middleton_Murry`}.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Person`; P_Q contains neither `dbo:spouse` nor `dbo:deathCause`.
3	Covers ($\mathcal{S}_Q, Q$)	covered=$\emptyset$; uncovered={*wife* -> no observed `dbo:spouse`, *die* -> no observed `dbo:deathCause`}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}, q_1 \to q_2$); q_1="Who was John Middleton Murry's wife?"; q_2="Why did [q_1] die?"; $M \leftarrow \emptyset$.
5	Ground(q_1, M)	$\hat{q}_1$="Who was John Middleton Murry's wife?".
6	EntityLink($\hat{q}_1$)	E_{q_1}={e_0}.
7	SchemaProject ($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Person`; P_{q_1} does not contain `dbo:spouse`.
8	Covers ($\mathcal{S}_{q_1}, \hat{q}_1$)	**false**; covered=$\emptyset$; uncovered={*wife* -> no coverage-compatible `dbo:spouse`}; next branch=**TEXT**.
9	TextRAG ($\hat{q}_1, E_{q_1}, \mathcal{T}$)	LLM answer over retrieved text; $\mathcal{T}_{q_1,\text{top}} \ni$ *John Middleton Murry*; a_{q_1}=*Katherine Mansfield*; $M[q_1] \leftarrow$*Katherine Mansfield*.
10	Ground(q_2, M)	$\hat{q}_2$="Why did Katherine Mansfield die?".
11	EntityLink($\hat{q}_2$)	E_{q_2}={e_1=`dbr:Katherine_Mansfield`}.
12	SchemaProject ($E_{q_2}, \hat{q}_2$)	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2} does not contain `dbo:deathCause`.
13	Covers ($\mathcal{S}_{q_2}, \hat{q}_2$)	**false**; covered=$\emptyset$; uncovered={*die* -> no coverage-compatible `dbo:deathCause`}; next branch=**TEXT**.
14	TextRAG ($\hat{q}_2, E_{q_2}, \mathcal{T}$)	LLM answer over retrieved text; $\mathcal{T}_{q_2,\text{top}} \ni$ *Katherine Mansfield*; a_{q_2}=*tuberculosis*; $M[q_2] \leftarrow$*tuberculosis*.
15	Synthesize(Q, M)	LLM synthesis; a=*Tuberculosis*.

3 Prompt Catalogue

For each prompt, we report its identifier, execution stage, and condition of use. The boxes below render runtime templates with placeholders; dynamic values such as candidate lists and retrieved passages are inserted at execution time.

Table 4: Prompt catalogue summary.

ID	Stage	Prompt	Condition of use
P1	Entity linking	Entity candidate disambiguation	Only when fallback entity search returns multiple DBpedia candidates for an unlinked mention.
P2	Routing	Schema-constrained property extraction	Only when deterministic keyword/schema matching is inconclusive; candidate properties are passed back to deterministic routing and ASK checks.
P3	Decomp.	Question decomposition	Used after the direct symbolic shortcut fails and decomposition is needed.
P4	Decomp.	Early stopping / accumulated facts	Used during decomposition to decide whether accumulated facts already answer the original question.
P5	SPARQL	SPARQL generation	Used for direct SPARQL and routed sub-question SPARQL.
P6	SPARQL	SPARQL repair	Used once when a generated SPARQL query errors or returns empty results.
P7	SPARQL	SPARQL result verbalization	Used to convert SPARQL result rows into a concise final answer.
P8	Text fallback	Whole-question answer prompt	Used when a whole question is routed to text or symbolic execution cannot answer.
P9	Text fallback	Sub-question answer prompt	Used when an individual decomposed sub-question is routed to text.
P10	Synthesis	Final answer synthesis	Used to synthesize the final answer from resolved sub-question facts.

continued on next page

ID	Stage	Prompt	Condition of use
J1	Judge	LLM-as-judge system and user prompts	Used only for evaluation records not handled by deterministic empty-answer, exact-alias, or containment pre-checks.

3.1 Runtime KARMA prompts

The prompts below are the active LLM templates used by the reported KARMA runtime. Prompt P2 is a fallback property-enrichment prompt: it proposes candidate DBpedia properties, but the route is still determined by schema matching and ASK verification.

P1. Entity candidate disambiguation

```
Given this question: "{question}"
Which entity best matches the mention "{mention}"?

{candidate_list}

Reply with ONLY the number (1-{num_candidates}) of the best match, or "none" if no candidate fits.
```

P2. Schema-constrained property extraction

```
Given this question and DBpedia schema, which properties are needed to answer it?

{schema}

QUESTION: {question}

Output a comma-separated list of property names (no dbo:/dbp: prefix).
Only output properties listed above. For 2-hop paths (e.g., "director's birthdate"),
list both: director, birthDate
If no properties can help, output: NONE

Properties:
```

P3. Decomposition prompt

```
Decompose this multi-hop question into independent sub-questions.

QUESTION: {question}
ENTITIES: {entity_names}
SCHEMA SUMMARY: {schema_summary}
{type_hint}
Rules:
- Each sub-question should ask for ONE fact
- If a sub-question needs the answer from another, mark the dependency
- For comparison questions: extract the same property for each entity (parallel, no dependencies)
- For bridge questions: first find the intermediate entity, then ask about its property
- If the question is simple (single fact lookup), return exactly 1 sub-question with no dependencies

Return JSON:
{
  "sub_questions": ["sub-Q1 text", "sub-Q2 text", ...],
  "dependencies": [[dependent_idx, dependency_idx], ...]
}

Example - Bridge: "When was the director of Inception born?"
{
  "sub_questions": ["Who directed Inception?", "When was [Q0] born?"],
  "dependencies": [[1, 0]]
}

Example - Comparison: "Who was born first, the director of X or Y?"
{
  "sub_questions": ["Who directed X?", "Who directed Y?",
    "When was [Q0] born?", "When was [Q1] born?"],
  "dependencies": [[2, 0], [3, 1]]
}

Example - Simple: "Who directed Inception?"
{
  "sub_questions": ["Who directed Inception?"],
  "dependencies": []
}

Example - Nested relative clause (3-hop): "When was the region north of where X is located created?"
{
```

```
  "sub_questions": ["Where is X located?", "What region is immediately north of [Q0]?", "When was [Q1] created?"],
  "dependencies": [[1, 0], [2, 1]]
}

Example - Passive voice: "Who is played by the director of Film X in Film Y?"
{
  "sub_questions": ["Who directed Film X?", "What role did [Q0] play in Film Y?"],
  "dependencies": [[1, 0]]
}

Example - Possessive chain: "What is the capital of the country where X's creator was born?"
{
  "sub_questions": ["Who created X?", "In which country was [Q0] born?", "What is the capital of [Q1]?"],
  "dependencies": [[1, 0], [2, 1]]
}

Additional rules for complex questions:
- Preserve active/passive voice direction: "played BY X" -> ask "What did [Q0] play?" NOT "Who played [Q0]?"
- For possessive chains (X's Y's Z), decompose from innermost entity outward
- Each sub-question MUST retain qualifying context (where/who/that clauses) from the original

JSON only, no explanation:
```

P4. Early stopping prompt

```
We pre-parsed the question into a list of dependencies, and the dependencies are sorted in a topological order, below
     is the question, the information summary, and the decomposed dependencies:

Question: {question}

Available Information:
{info_summary}

Decomposed dependencies:
{dependencies}

Current dependency to be answered:
{current_dependency}

Please analyze the question and the information summary, and the decomposed dependencies, and answer the following
     questions:
Please analyze:
1. Can the question be answered completely with this information? (Yes/No)
2. Summarize our current understanding based on available information.

Please format your response as a JSON object with these keys:
- "can_answer": boolean
- "current_understanding": string
```

P5. SPARQL generation prompt

```
You are a SPARQL expert for DBpedia. Generate a query to answer the question.

{schema}
{known_facts_block}
QUESTION: {question}

RULES:
1. Use the EXACT entity URIs provided in the schema
2. For multi-hop queries, chain triple patterns using dbo: properties (they return URIs, not strings).
   Use the "2-Hop chains" section in the schema to identify valid chains.
   Example: if schema shows dbo:director -> Person: birthDate, write:
   ?film dbo:director ?person . ?person dbo:birthDate ?date .
3. Prefer dbo: properties for multi-hop joins; only use dbp: for single-hop leaf values
4. Always add LIMIT 100
5. For comparison questions (who/which was born first, etc.), return data for BOTH entities so results can be compared
6. If an entity URI contains special characters (apostrophes, commas, parentheses), use full IRI syntax <http://
     dbpedia.org/resource/...> instead of dbr: prefix
7. If known facts provide entity URIs, use those URIs directly in your query
8. Generate ONLY the SPARQL query, no explanation
```

P6. SPARQL repair prompt

```
Your previous SPARQL query returned {problem}.

Previous query:
{sparql}

Fix the query using the schema below.

{schema}
```

```
QUESTION: {question}

Generate ONLY the corrected SPARQL query:
```

P7. SPARQL result verbalization prompt

```
Given the question and SPARQL query results from DBpedia, provide the answer.
Base your answer ONLY on the SPARQL results shown below, not on your own knowledge.
For comparison questions: if data is missing for one of the entities being compared, say "could not determine" instead
      of guessing.

Question: {question}
SPARQL Results:
{results}

End your response with FINAL ANSWER: <your answer>
```

P8. Whole-question text fallback prompt

```
Answer this question using the sources below.

Question: {question}

{kg_facts}Wikipedia text:
{context}

Instructions:
- Knowledge graph facts (if present) are verified structured data -- trust them
- Use Wikipedia text to fill gaps not covered by KG facts
- Show brief step-by-step reasoning
- End with exactly: FINAL ANSWER: <answer>
```

P9. Sub-question text fallback prompt

```
Answer this sub-question using the sources below.

Sub-question: {question}

{kg_facts}Wikipedia text:
{context}

Instructions:
- Knowledge graph facts (if present) are verified structured data -- trust them
- Use Wikipedia text to fill gaps not covered by KG facts
- Give a brief, factual answer
- End with exactly: FINAL ANSWER: <answer>
```

P10. Final answer synthesis prompt

```
You must give ONLY the direct answer in the most concise way possible.
DO NOT explain or provide any additional context.
If the answer is a simple yes/no, just say "Yes." or "No."
If the answer is a name, just give the name.
If the answer is a date, just give the date.
If the answer is a number, just give the number.
If the answer requires a brief phrase, make it as concise as possible.

IMPORTANT -- the sub-questions below form a reasoning chain:
- Earlier sub-questions find intermediate entities; later ones use those entities.
- The LAST sub-question's answer is usually what the original question asks for.
- Do NOT return an intermediate entity when a later sub-question has resolved the final answer.
- If a sub-question returns a list, pick the item that satisfies the condition in the original question.
- For yes/no questions with multiple sub-questions, combine ALL sub-answers (both must be true for "yes").
- If any sub-question is marked [UNKNOWN], the chain is incomplete. Do NOT fabricate
  a final answer by skipping the gap -- respond with: FINAL ANSWER: unknown

[Inserted only when is_comparison=True]
When comparing countries or nationalities:
- If answers contain city or state names, identify which country they belong to
- Compare at the country level, not city/state level
- Historical states (East Germany, Ottoman Empire) map to their modern successor

Question: {question}

Information Summary:
{info_summary}

End your response with exactly: FINAL ANSWER: <your answer>
```

3.2 Evaluation judge prompt

Prompt J1 is not a KARMA runtime prompt. It is used only by the evaluation script after deterministic checks have failed to decide the record: empty predictions are false, exact gold-or-alias matches are true, and a normalized gold-in-prediction containment pre-check can return true before the LLM call. The LLM judge applies the benchmark-equivalence rules shown below, including alias, containment, and selected location-normalization cases; it is stricter than free-form semantic similarity, but it is not an exact-match metric.

J1. LLM-as-judge prompt

```
System:
You are a strict evaluation judge for a QA benchmark. Determine if two answers refer to the exact same entity or fact.
      Precision matters more than recall -- when in doubt, answer no.

User:
Determine if the predicted answer is semantically equivalent to the gold answer.

Question: {question}
Gold answer: {gold}
Predicted answer: {pred}

Matching rules (answer YES):
- Same entity/fact with different surface forms (e.g. "July 16, 2013" vs "2013-07-16")
- Gold contained in a more specific predicted answer (e.g. "Munich" vs "Nymphenburg Palace, Munich")
- Added title/honorific/disambiguator (e.g. "Henry III" vs "Henry III, Holy Roman Emperor")
- Full formal name of same person (e.g. "Giovanni Mario" vs "Giovanni Mario de Candia")
- Administrative region containing the exact gold location (e.g. gold "Theni" vs pred "Theni district")
- Unambiguous well-known city with or without country (e.g. gold "Florence, Italy" vs pred "Florence")
- Nationality adjective matching country name (e.g. gold "Italy" vs pred "Italian")

Rejection rules (answer NO):
- Wrong, empty, or completely different entity
- Predicted is broader/vaguer than gold (e.g. gold "Taipei Medical University Hospital" vs pred "Taipei")
- Year only when gold has full date (e.g. gold "December 25, 1757" vs pred "1757")
- Related but distinct entities (e.g. gold "Hollywood" vs pred "Los Angeles")
- Same first name but different person (e.g. gold "Paris" vs pred "Paris Hilton")
- Predicted lists multiple answers and gold is NOT clearly among them
- Completely different location even if in same region (e.g. gold "Theni" vs pred "Mallingapuram")

First, explain in ONE sentence why the answers match or don't.
Then write your verdict on a new line: VERDICT: yes or VERDICT: no
```

4 DBpedia Coverage on the Benchmarks

Across the three benchmarks, one pattern recurs: entities are retrievable for 93–99% of questions, but the *relations* needed to chain them into an answer are sparse (20–37%). This gap is precisely what the coverage predicate $\textsc{Covers}(\mathcal{S}, Q)$ targets (§**??**), and it tracks the cross-benchmark drop in any-SPARQL engagement: 70.0% (2WikiMultiHopQA) → 42.6% (MuSiQue) → 17.3% (HotpotQA).

The three benchmarks expose this gap through different gold annotations, so we measure each with the strongest signal it affords. 2Wiki carries gold evidence triples $[s_i, p_i, o_i]$ per hop, enabling direct *predicate-completeness*: a question passes iff every gold predicate is asserted on the resolved subject. HotpotQA provides only supporting Wikipedia titles, so we fall back to *pair connectivity*; MuSiQue provides per-hop title and answer, so we use *chain connectivity*. Rates are therefore *not directly comparable across benchmarks*; for that we rely on the SPARQL engagement rate measured at run time (§**??**) as the unifying signal. Pair and chain connectivity are structural proxies for predicate-completeness: they check whether *some* predicate links the annotated entities, not whether the predicate the question requires is present, so we read the HotpotQA and MuSiQue numbers as lower bounds rather than measured predicate-completeness. All numbers are computed against DBpedia as-is, without graph construction. We follow classical KG completeness work [**?**, **?**] but report at the question and sub-question level: "does DBpedia provide the predicates each sub-question requires?"

Connectivity definitions. Let $\mathcal{E}_i = (e_1, \ldots, e_k)$ be the entity sequence annotated for question i.

Definition 1 (Pair Connectivity) *Two entities e_a, e_b are* pair-connected *in DBpedia if there exists a path $e_a \xrightarrow{p_1(p_2)?} e_b$ of 1 or 2 `dbo:` edges. A HotpotQA question (k=2) passes iff its single pair connects; a k>2 question passes iff every consecutive pair connects.*

Definition 2 (Chain Connectivity) *$\mathcal{E}_i$ is* chain-connected *iff for every consecutive pair (e_j, e_{j+1}), $j \in \{1, \ldots, k-1\}$, there exists a single direct predicate $p \in$ `dbo:` such that $(e_j, p, e_{j+1}) \in$ DBpedia. A MuSiQue question passes iff the entire sequence is chain-connected.*

Implementation notes. The connectivity checks above and the lexical matching used to compute the projected schema $\mathcal{S}(q)$ (§**??**, Def. **??**) share three restrictions worth making explicit. (i) *Namespace*: only predicates in the curated `dbo:` ontology count; `dbp:` infobox properties are excluded as infobox-level and not part of the curated ontology. (ii) *Path shape*: only direct one-hop assertions (e, p, o) are counted; SPARQL property paths (p^+, p^*, $\hat{}p$) are not. (iii) *Aliases*: `owl:sameAs` redirects are followed and `rdfs:label` variants are treated as equivalent under the same token-level normalization used by the lexical relevance check $p \sim q$ in $\mathcal{S}(q)$.

Table 5: Coverage rates per benchmark (n=1000, question-level). *Structural* is benchmark-specific: predicate-completeness for 2Wiki (Def. not needed–direct from gold triples), pair connectivity for HotpotQA (Def. 1), chain connectivity for MuSiQue (Def. 2). *Answer grounding* requires the gold answer to match a DBpedia predicate value; not applicable to 2Wiki (see footnote).

Benchmark	Entity	Structural	Answer grounding
2Wiki	93.6%	36.8% (pred.-compl.)	n/a†
HotpotQA	98.0%	26.8%‡ (pair conn.)	43.8%‡‡
MuSiQue	99.8%	20.9% (chain conn.)	23.1%

† String-exact value match (11.9% raw) is dominated by date/label format mismatches, not missing KG values; predicate-completeness is therefore the primary KG-answerable metric.

‡ The 26.8% figure is 268/1000 (whole-benchmark denominator). Pair connectivity holds only for bridge questions (268/811 = 33.0% when restricted to bridges); the 189 comparison questions contribute zero connected pairs, since the same-type juxtaposition does not require an ontological relation. The 98% entity rate counts all questions.

‡‡ Strict answer grounding 303/692: excludes 225 entity-as-answer, 63 yes/no, 20 unresolved (see HotpotQA paragraph).

2WikiMultiHopQA: predicate gaps dominate entity gaps (8.9×). 36.8% of questions are predicate-complete; 63.2% are not. Among the 632 failures, 568 fail on missing *predicates* and only 64 on missing *entities*: predicate gaps outnumber entity gaps 8.9×, motivating predicate-based routing. We can only measure this gap directly on 2Wiki, since HotpotQA and MuSiQue lack gold predicates; the high entity rates on those two benchmarks (98.0%, 99.8%) are consistent with predicate-bound rather than entity-bound coverage. Predicate-completeness varies sharply with question type: bridge_comparison 60.0% (n=235), comparison 32.8% (n=244), inference 31.5% (n=108), compositional 27.4% (n=413) (Appendix Table 8)—the same ranking observed for symbolic routing gains in Table **??**.

HotpotQA: bridges only. Pair connectivity concentrates entirely on bridge questions (268/811 = 33.0%); the 189 comparison questions contribute *zero* connected pairs because

Table 6: Per-run breakdown of end-to-end results (main Table **??**) on 2Wiki, HotpotQA, and MuSiQue (1,000 questions each). r1 / r2: run 1 / run 2 (%). $|\Delta|$: absolute per-run drift $|r1 - r2|$. Cont.: substring containment after normalization. Acc.: judge accuracy (Gemma 3 12B). KARMA_{KG}: SPARQL-only variant.

		Cont. (%)			**Acc. (%)**		
System	**Benchmark**	r1	r2	$\|\Delta\|$	r1	r2	$\|\Delta\|$
LogicRAG	2Wiki	69.2	69.0	0.2	74.5	74.9	0.4
	HotpotQA	58.7	59.9	1.2	78.8	79.2	0.4
	MuSiQue	37.6	38.0	0.4	50.6	52.0	1.4
LinearRAG	2Wiki	76.3	76.3	0.0	84.0	83.6	0.4
	HotpotQA	65.9	66.3	0.4	86.9	87.5	0.6
	MuSiQue	37.2	37.8	0.6	51.6	51.6	0.0
KARMA	2Wiki	83.1	84.5	1.4	89.4	90.4	1.0
	HotpotQA	63.9	64.1	0.2	86.1	85.3	0.8
	MuSiQue	54.8	50.0	4.8	65.6	62.5	3.1
KARMA_{KG}	2Wiki	36.7	31.9	4.8	39.6	34.0	5.6
	HotpotQA	3.6	3.9	0.3	5.5	5.5	0.0
	MuSiQue	2.5	2.4	0.1	2.9	2.9	0.0

same-type juxtaposition does not require an ontological relation in DBpedia. The strict answer-grounding rate 303/692 = 43.8% excludes 225 entity-as-answer, 63 yes/no, and 20 unresolved questions.[1] Entity access is high while ontological links are sparse, consistent with reduced symbolic routing on HotpotQA.

MuSiQue: per-hop grounding can exceed chain connectivity. A hop answer may exist as a DBpedia predicate value even when the full entity chain cannot be traversed end-to-end. As a result, answer grounding (23.1%) can exceed chain connectivity (20.9%). Although chain connectivity is too sparse to support a single top-level SPARQL query (2.4% direct direct answering), decomposition into sub-questions raises SPARQL engagement to 42.6% (§**??**).

5 Additional analyses

5.1 Per-run stability

We report main Table **??** from a single run (run 1) to keep headline numbers uncluttered. All systems are evaluated over two independent runs using the same configuration. Table 6 reports run 1, run 2, and the absolute per-run drift $|\Delta|=|\text{r1}-\text{r2}|$ for both metrics on all three benchmarks. Across the hybrid and text-RAG baselines, maximum drift is 3.1% for judge accuracy (KARMA, MuSiQue) and 4.8% for substring containment (KARMA, MuSiQue); drift on the other cells stays at or below 1.4%. KARMA_{KG} is more variable on 2WikiMultiHopQA (5.6% Acc., 4.8% Cont.) but stable on HotpotQA and MuSiQue ($\leq 0.3\%$ on both metrics). KARMA holds the top rank across both runs on every benchmark; on MuSiQue Acc., the second and third positions (LinearRAG, LogicRAG) swap within a 1.4% margin.

[1] A looser variant that additionally credits the 225 entity-as-answer questions (whose gold answers are themselves resolvable entity titles) reaches 528/1000 = 52.8%. We report the strict rate as the primary metric because it isolates the harder setting in which the answer must be retrieved from a literal predicate value.

Table 7: Judge accuracy: Gemma 3 12B (main results) vs. Claude Sonnet 4 (single Claude pass over run-1 predictions, %).

Benchmark	Judge	KARMA	LogicRAG	LinearRAG
2Wiki	Gemma 3 12B	89.4	74.5	84.0
	Claude Sonnet 4	88.4	73.3	82.7
HotpotQA	Gemma 3 12B	86.1	78.8	86.9
	Claude Sonnet 4	81.3	75.1	83.7
MuSiQue	Gemma 3 12B	65.6	50.6	51.6
	Claude Sonnet 4	58.5	45.9	44.2

Table 8: Judge accuracy (%) by question type across 2Wiki, HotpotQA, and MuSiQue (1,000 questions per benchmark), using Gemma 3 12B as judge. NB: number of queries. Values are mean ± half-range across two runs. Best per row in **bold**.

Benchmark	Type	NB	KARMA	LogicRAG	LinearRAG
2Wiki	bridge_comp.	235	93.4±1.5	72.8±0.0	**93.8**±0.2
	comparison	244	95.5±0.4	92.8±0.2	**96.7**±0.0
	compositional	413	**83.4**±0.8	62.1±0.4	71.2±0.7
	inference	108	**94.4**±0.9	85.6±0.5	80.6±0.0
HotpotQA	bridge	811	84.6±0.2	77.1±0.1	**86.9**±0.0
	comparison	189	**90.2**±1.3	87.6±0.8	88.1±1.3
MuSiQue	2-hop	518	**70.6**±0.9	61.9±0.7	58.4±0.9
	3-hop	316	**68.4**±2.2	47.2±1.3	48.3±0.8
	4-hop	166	35.5±2.4	26.2±0.3	**36.8**±1.2

5.2 Second judge: Claude vs. Gemma

The main results (Table **??**) use Gemma 3 12B as judge. To check that system rankings are robust to judge choice, we cross-check with Claude Sonnet 4 over the run-1 predictions of all three systems. Claude is uniformly stricter, with lower absolute scores across the nine cells. On 2Wiki the system ranking is preserved. On MuSiQue, KARMA remains first (+12.6% under Claude) while LogicRAG and LinearRAG, separated by only 1.0% under Gemma, swap under Claude. On HotpotQA the gap between KARMA and LinearRAG widens to 2.4%; ranking vs. LogicRAG is preserved. Table 7 reports the comparison.

5.3 Per-type accuracy breakdown

Table **??** in the main text reports per-type judge accuracy. Table 8 here gives the numeric values with run-to-run half-ranges, and also backs the predicate-completeness ranking by question type discussed in Appendix 4.